

Djoko Studio — Project Idea

MVP d'un studio d'interview vidéo résilient aux connexions instables

Djoko Christian

13 juin 2026

Table des matières

Djoko Studio — Project Idea	3
1. Vision du projet	3
2. Problème identifié	3
3. Positionnement	3
4. Utilisateurs cibles	4
5. Principes produit	4
5.1 Local-first recording	4
5.2 Résilience réseau	5
5.3 Pistes séparées	5
5.4 Simplicité d'usage	5
5.5 Ambition technique	5
6. MVP global	6
7. Version 0.1 — Recording Core	6
Objectif de la v0.1	6
Fonctionnalités incluses en v0.1	6
7.1 Compte host	6
7.2 Studio personnel	7
7.3 Session d'enregistrement	7
7.4 Invitation guest par lien	7
7.5 Appel vidéo live 1 host + 1 guest	8
7.6 Mode solo recording	8
7.7 Enregistrement local	8
7.8 Pistes séparées	9
7.9 Upload progressif par chunks	9
7.10 Sauvegarde locale avant upload	10
7.11 Reprise automatique d'upload	10
7.12 Écran automatique de récupération	10
7.13 Reconnexion automatique	10
7.14 Indicateurs obligatoires	11
7.15 Visibilité host sur l'upload du guest	11
7.16 Avertissement avant fermeture	11
7.17 Dashboard host	11
7.18 Export final 1080p YouTube	12

7.19 Téléchargement des pistes brutes	12
7.20 Confidentialité par défaut	12
Exclusions de la v0.1	13
8. Version 0.2 — Editing Core	13
Objectif de la v0.2	13
Fonctionnalités incluses en v0.2	13
8.1 Transcription automatique	13
8.2 Sous-titres automatiques	14
8.3 Édition par texte	14
8.4 Timeline logique	14
8.5 Rendu vidéo édité	15
8.6 Export transcript	15
Exclusions de la v0.2	15
9. Version 0.3 — Screen Share Core	15
Objectif de la v0.3	15
Fonctionnalités incluses en v0.3	16
9.1 Partage d'écran pendant l'appel	16
9.2 Intégration dans la vidéo finale	16
9.3 Segments de screen share	16
Exclusions de la v0.3	16
10. Architecture fonctionnelle pressentie	17
10.1 Frontend Studio	17
10.2 Signaling Service	17
10.3 Backend API	17
10.4 Upload Service	18
10.5 Object Storage	18
10.6 Database	18
10.7 Workers	18
11. Contraintes de qualité	19
11.1 Qualité vidéo cible	19
11.2 Résilience	19
11.3 Observabilité	19
12. Ambition business	20
13. Pourquoi ce projet est pertinent pour apprendre l'architecture	20
14. Décisions d'architecture à documenter plus tard	21
15. Définition courte du projet	22

Djoko Studio — Project Idea

1. Vision du projet

Djoko Studio est une plateforme web d'enregistrement d'interviews vidéo à distance, pensée pour les créateurs, entrepreneurs, podcasteurs et intervieweurs qui veulent produire du contenu de qualité même lorsque la connexion Internet est instable.

L'objectif n'est pas de créer un simple outil d'appel vidéo. L'objectif est de créer un **studio d'enregistrement résilient**, où l'appel en direct peut s'adapter à la connexion, mais où la qualité finale de l'enregistrement reste propre, stable et exploitable.

La promesse principale du produit est :

Même si la connexion est instable, l'enregistrement local reste sécurisé, l'upload reprend automatiquement, et les pistes finales restent disponibles.

Djoko Studio vise d'abord les interviews à distance entre **un host et un invité**, avec un workflow simple : créer un studio, inviter une personne, enregistrer, uploader automatiquement, récupérer les pistes et exporter une vidéo finale prête pour YouTube.

2. Problème identifié

Les outils existants comme Riverside, StreamYard, Zoom ou Google Meet permettent déjà d'enregistrer des interviews à distance. Mais dans un contexte où la connexion Internet peut être instable, notamment en Afrique, plusieurs problèmes apparaissent :

- l'appel peut couper brutalement ;
- l'utilisateur peut être expulsé de la session ;
- certaines parties de l'enregistrement peuvent être perdues ;
- l'upload peut échouer ou rester bloqué ;
- le host ne sait pas toujours si l'invité a bien terminé l'upload ;
- la qualité finale peut dépendre fortement de la connexion ;
- les plans gratuits sont souvent très limités ;
- les outils existants ne sont pas toujours pensés pour des créateurs avec des contraintes réseau fortes.

Djoko Studio veut attaquer un problème précis :

Comment permettre à un créateur d'enregistrer une interview vidéo propre, même lorsque le réseau est instable ?

3. Positionnement

Djoko Studio ne cherche pas à être un clone complet de Riverside.

Le positionnement du MVP est plus ciblé :

Un studio d'interview web, local-first, résilient aux coupures réseau, avec enregistrement local, upload par chunks, reprise automatique, pistes séparées et export YouTube 1080p.

La différenciation principale n'est pas d'avoir plus de fonctionnalités que Riverside, mais d'être meilleur sur un point précis :

La fiabilité de l'enregistrement et de l'upload dans des conditions réseau difficiles.

4. Utilisateurs cibles

Les premiers utilisateurs visés sont :

- créateurs de contenu ;
- podcasteurs ;
- intervieweurs ;
- entrepreneurs ;
- formateurs ;
- journalistes indépendants ;
- créateurs africains travaillant avec des invités à distance ;
- personnes qui veulent enregistrer des vidéos solo ou des interviews sans dépendre d'un outil trop limité en version gratuite.

Le MVP doit supporter deux cas d'usage principaux :

1. Interview à distance

- 1 host connecté avec un compte ;
- 1 invité qui rejoint via un lien sans créer de compte.

2. Enregistrement solo

- le host peut enregistrer seul une vidéo personnelle ;
 - utile pour des introductions, capsules, vidéos YouTube, formations ou messages personnels.
-

5. Principes produit

Djoko Studio repose sur plusieurs principes forts.

5.1 Local-first recording

L'enregistrement ne doit pas dépendre uniquement du serveur.

Chaque participant enregistre localement ses propres pistes audio et vidéo dans le navigateur. Les fichiers sont ensuite uploadés progressivement vers le cloud.

Cela permet de séparer deux choses :

- la qualité de l'appel en direct ;
- la qualité du fichier final enregistré.

Même si l'appel live est instable, le fichier final peut rester propre.

5.2 Résilience réseau

Le système doit être conçu pour gérer :

- coupure temporaire de connexion ;
- upload interrompu ;
- reconnexion automatique ;
- page rechargée ;
- invité qui tente de quitter avant la fin de l'upload ;
- réseau lent ;
- upload qui reprend plus tard.

La priorité est de ne pas perdre les données enregistrées.

5.3 Pistes séparées

Chaque participant doit avoir ses propres pistes :

- host audio ;
- host vidéo ;
- guest audio ;
- guest vidéo.

Cela permet une meilleure qualité de post-production, une édition plus propre et une meilleure récupération en cas de problème sur une piste.

5.4 Simplicité d'usage

L'utilisateur ne doit pas sentir toute la complexité technique.

Le workflow doit rester simple :

1. créer un studio ;
2. inviter un guest ;
3. démarrer l'enregistrement ;
4. terminer l'enregistrement ;
5. attendre la synchronisation ;
6. éditer automatiquement la vidéo via la transcription ;
7. télécharger la vidéo finale, la vidéo éditée ou les pistes brutes.

5.5 Ambition technique

Le projet est volontairement ambitieux. Il doit servir de terrain d'apprentissage pour :

- architecture logicielle ;
- system design ;
- WebRTC ;
- bases de données ;
- stockage média ;
- upload résilient ;
- workers ;
- traitement vidéo ;
- transcription ;

- édition vidéo ;
 - déploiement ;
 - observabilité ;
 - migrations de données ;
 - scalabilité.
-

6. MVP global

Le MVP de Djoko Studio est découpé en trois :

- **v0.1 — Recording Core**
- **v0.2 — Editing Core**
- **v0.3 — Screen Share Core**

L'objectif est de construire progressivement un produit sérieux, sans mélanger toutes les complexités dès la première phase.

7. Version 0.1 — Recording Core

Objectif de la v0.1

Construire le cœur du produit : enregistrer une interview vidéo à distance de manière fiable, avec enregistrement local, upload progressif, reprise automatique, pistes séparées et export final 1080p pour YouTube.

La v0.1 doit prouver que Djoko Studio peut tenir sa promesse principale :

Même si la connexion est instable, l'enregistrement reste sauvegardé et récupérable.

Fonctionnalités incluses en v0.1

7.1 Compte host

Le host doit pouvoir créer un compte.

Le compte est nécessaire pour :

- créer des studios ;
- gérer ses enregistrements ;
- retrouver son historique ;
- gérer ses quotas plus tard ;
- permettre une future facturation ;
- sécuriser les fichiers ;
- associer chaque recording à son propriétaire.

L'invité, lui, ne crée pas de compte.

7.2 Studio personnel

Le host peut créer un ou plusieurs studios.

Un studio représente un espace d'enregistrement.

Exemple :

- Studio "Interviews Tech Afrique"
- Studio "LOGOS"
- Studio "Podcast Business"
- Studio "Vidéos solo"

Chaque studio peut contenir plusieurs sessions d'enregistrement.

7.3 Session d'enregistrement

Le host peut créer une session dans un studio.

Une session représente une interview ou un enregistrement solo.

Une session contient :

- un titre ;
- un statut ;
- une date de création ;
- un host ;
- éventuellement un guest ;
- les pistes enregistrées ;
- les chunks uploadés ;
- les exports générés.

Statuts possibles :

- draft ;
 - waiting_guest ;
 - ready ;
 - recording ;
 - uploading ;
 - processing ;
 - completed ;
 - failed.
-

7.4 Invitation guest par lien

Le host peut générer un lien d'invitation.

L'invité rejoint la session sans compte.

Le lien doit être sécurisé et limité à une session précise.

L'invité doit pouvoir :

- entrer son nom ;
 - autoriser caméra et micro ;
 - rejoindre la room ;
 - participer à l'appel ;
 - enregistrer localement ;
 - uploader automatiquement ses fichiers.
-

7.5 Appel vidéo live 1 host + 1 guest

Le MVP supporte uniquement :

- 1 host ;
- 1 invité.

Pas de multi-invités en v0.1.

L'appel live sert à permettre la conversation en temps réel.

Le live doit être aussi fluide que possible, mais il peut adapter sa qualité selon le réseau.

Principe important :

Le live peut baisser en qualité, mais l'enregistrement final doit rester en haute qualité locale.

7.6 Mode solo recording

Le host peut enregistrer seul.

Ce mode permet de créer :

- vidéos personnelles ;
- intros ;
- capsules YouTube ;
- formations ;
- messages ;
- tests caméra/micro.

Dans ce mode, il n'y a pas d'invité ni de WebRTC nécessaire entre deux personnes.

7.7 Enregistrement local

Chaque participant enregistre localement ses propres flux.

Pour une interview normale :

- le host enregistre localement son audio ;
- le host enregistre localement sa vidéo ;
- le guest enregistre localement son audio ;

- le guest enregistre localement sa vidéo.

L'enregistrement local est prioritaire sur l'upload cloud.

Cela veut dire que le système doit d'abord s'assurer que le segment est sauvegardé localement avant de tenter l'upload.

7.8 Pistes séparées

Le système doit produire des pistes séparées :

- host audio ;
- host vidéo ;
- guest audio ;
- guest vidéo.

Cela permettra plus tard :

- édition plus propre ;
 - amélioration audio ;
 - correction d'un participant sans affecter l'autre ;
 - synchronisation ;
 - export vidéo final ;
 - récupération partielle si une piste pose problème.
-

7.9 Upload progressif par chunks

L'enregistrement ne doit pas être uploadé en un seul gros fichier à la fin.

Le navigateur doit produire des chunks réguliers et les uploader progressivement.

Chaque chunk doit avoir :

- un identifiant ;
- un numéro d'ordre ;
- une session associée ;
- une piste associée ;
- une taille ;
- un checksum ou mécanisme de validation ;
- un statut d'upload.

Statuts possibles d'un chunk :

- pending ;
 - uploading ;
 - uploaded ;
 - failed ;
 - retrying ;
 - verified.
-

7.10 Sauvegarde locale avant upload

Chaque chunk doit être sauvegardé localement avant l'upload.

Objectif :

Si la connexion tombe, les chunks non uploadés restent présents dans le navigateur.

Le système doit être capable de reprendre l'upload quand la connexion revient.

7.11 Reprise automatique d'upload

Si l'upload échoue, le système doit réessayer automatiquement.

Cas à gérer :

- réseau lent ;
- perte de connexion ;
- refresh de page ;
- fermeture accidentelle ;
- retour plus tard dans la même session ;
- upload partiel.

Le système doit reprendre à partir des chunks manquants, pas recommencer tout l'upload depuis zéro.

7.12 Écran automatique de récupération

Même si l'utilisateur ne voit pas une page "récupération" dans le menu, l'application doit détecter automatiquement les enregistrements non synchronisés.

Si des chunks non uploadés sont trouvés, l'utilisateur doit voir un écran simple :

Nous avons trouvé un enregistrement non synchronisé.

Upload cloud : 63%.

Veuillez laisser cette page ouverte pour finaliser la sauvegarde.

Cette fonctionnalité est incluse dès la v0.1.

7.13 Reconnexion automatique

Si la connexion coupe pendant l'interview :

- l'enregistrement local continue autant que possible ;
- l'application affiche l'état de l'enregistrement ;
- l'application tente de reconnecter le participant à la room ;
- l'upload reprend quand le réseau revient.

Le message côté utilisateur doit être clair :

Connexion perdue. L'enregistrement local continue. Tentative de reconnexion...

7.14 Indicateurs obligatoires

L'interface doit afficher deux informations simples :

Enregistrement local : OK

Upload cloud : 42%

Ces deux indicateurs sont essentiels.

Ils permettent au host et au guest de comprendre si l'enregistrement est sécurisé.

7.15 Visibilité host sur l'upload du guest

Le host doit voir l'état d'upload du guest.

Exemple :

Host upload : 100%

Guest upload : 74%

Cela évite que le host pense que tout est terminé alors que l'invité n'a pas encore fini d'uploader.

7.16 Avertissement avant fermeture

Si un participant essaie de fermer la page alors que l'upload n'est pas terminé, l'application doit l'avertir.

Message possible :

Votre enregistrement n'est pas encore totalement sauvegardé dans le cloud.

Upload cloud : 72%.

Si vous quittez maintenant, l'upload reprendra plus tard, mais il est préférable de laisser cette page ouverte.

Le but n'est pas de bloquer agressivement, mais d'éviter les pertes par erreur.

7.17 Dashboard host

Le host doit avoir un dashboard simple avec :

- liste de ses studios ;
- liste de ses sessions ;
- statut de chaque session ;
- upload progress ;
- bouton télécharger ;

- bouton voir les pistes ;
 - bouton voir export final ;
 - bouton supprimer.
-

7.18 Export final 1080p YouTube

Le système doit générer automatiquement une vidéo finale 16:9 en 1080p stable.

Format cible :

1920x1080

16:9

MP4

adapté YouTube

Layout simple :

- host à gauche ;
- invité à droite.

Le host doit pouvoir télécharger directement cette vidéo sans passer par l'éditeur.

7.19 Téléchargement des pistes brutes

Le host doit pouvoir télécharger :

- vidéo brute du host ;
- audio brut du host ;
- vidéo brute du guest ;
- audio brut du guest ;
- vidéo finale 1080p.

Cela permet de faire un montage externe si besoin.

7.20 Confidentialité par défaut

Les recordings sont privés par défaut.

Seul le host propriétaire de la session peut accéder :

- aux pistes ;
- à l'export ;
- aux fichiers ;
- à la session ;
- aux futures transcriptions.

Les guests n'ont pas accès au dashboard du host.

Exclusions de la v0.1

La v0.1 n'inclut pas :

- multi-invités ;
 - mobile ;
 - livestream ;
 - chat ;
 - markers ;
 - téléprompteur ;
 - media board ;
 - édition vidéo ;
 - édition par transcription ;
 - Magic Clips ;
 - formats TikTok/Reels ;
 - équipes ;
 - publication directe YouTube ;
 - paiement ;
 - billing ;
 - screen share.
-

8. Version 0.2 — Editing Core

Objectif de la v0.2

Ajouter une couche d'édition simple basée sur la transcription.

Le but est de permettre au host de modifier une vidéo en supprimant du texte dans la transcription.

Principe :

Supprimer une phrase dans la transcription coupe automatiquement le segment correspondant dans la vidéo.

Fonctionnalités incluses en v0.2

8.1 Transcription automatique

Après traitement de l'enregistrement, le système génère une transcription.

La transcription doit inclure :

- texte ;
- timestamps ;
- séparation par segments ;
- idéalement identification du speaker.

Exemple :

[00:01:12 - 00:01:18] Host : Peux-tu te présenter rapidement ?
[00:01:19 - 00:01:35] Guest : Oui, je suis fondateur de...

8.2 Sous-titres automatiques

Le système doit générer des fichiers de sous-titres.

Formats possibles :

- SRT ;
- VTT.

Ces sous-titres pourront être utilisés pour :

- YouTube ;
 - lecteur interne ;
 - export final ;
 - édition.
-

8.3 Édition par texte

Le host peut supprimer des segments directement dans la transcription.

Exemple :

Le host sélectionne :

[00:12:01 - 00:12:15] Euh... attends, je reprends ma question.

Puis clique sur "Supprimer".

Le système marque ce segment comme coupé.

8.4 Timeline logique

Le système doit maintenir une timeline d'édition.

Il ne doit pas détruire les fichiers originaux.

Il doit stocker une liste de cuts.

Exemple :

Cut 1 : 00:12:01 → 00:12:15

Cut 2 : 00:25:04 → 00:25:30

Cut 3 : 00:48:10 → 00:49:02

Cela permet de rendre une nouvelle vidéo sans perdre l'original.

8.5 Rendu vidéo édité

Après modification de la transcription, le host peut générer une nouvelle vidéo.

Le système produit :

final-edited-1080p.mp4

L'original reste disponible.

8.6 Export transcript

Le host peut télécharger :

- transcription complète ;
 - transcription éditée ;
 - sous-titres SRT/VTT.
-

Exclusions de la v0.2

La v0.2 n'inclut pas :

- Magic Clips automatiques ;
 - détection automatique des meilleurs moments ;
 - suppression automatique des silences ;
 - correction audio avancée ;
 - noise removal avancé ;
 - publication directe ;
 - formats verticaux ;
 - captions stylisées complexes.
-

9. Version 0.3 — Screen Share Core

Objectif de la v0.3

Ajouter le partage d'écran pour enrichir les interviews, surtout les interviews tech, business, produit ou formation.

Le screen share permet au host ou au guest de montrer :

- un site web ;
- une application ;
- un dashboard ;
- une présentation ;
- un document ;
- une démo produit ;
- un prototype ;
- une interface.

Fonctionnalités incluses en v0.3

9.1 Partage d'écran pendant l'appel

Le host ou le guest peut partager son écran pendant l'interview.

Le partage d'écran est visible en live par l'autre participant.

9.2 Intégration dans la vidéo finale

Le screen share doit apparaître dans la vidéo finale.

Layout possible :

- écran partagé en grand ;
- host en petite fenêtre ;
- guest en petite fenêtre.

Quand aucun écran n'est partagé, le layout revient au mode interview classique :

- host à gauche ;
 - guest à droite.
-

9.3 Segments de screen share

Le système doit savoir à quel moment le screen share commence et s'arrête.

Exemple :

Screen share started : 00:14:22

Screen share stopped : 00:26:10

Cela permet au moteur d'export de choisir automatiquement le bon layout.

Exclusions de la v0.3

La v0.3 n'inclut pas encore :

- piste brute séparée du screen share ;
 - annotation live ;
 - dessin sur l'écran ;
 - contrôle à distance ;
 - multi-screen ;
 - présentation slides avancée.
-

10. Architecture fonctionnelle pressentie

Djoko Studio sera composé de plusieurs grands blocs.

10.1 Frontend Studio

Responsabilités :

- interface host ;
 - interface guest ;
 - permissions caméra/micro ;
 - live preview ;
 - WebRTC ;
 - local recording ;
 - stockage local temporaire ;
 - upload manager ;
 - indicateurs recording/upload ;
 - édition texte plus tard.
-

10.2 Signaling Service

Responsabilités :

- créer une room live ;
 - connecter host et guest ;
 - gérer présence ;
 - gérer reconnexion ;
 - échanger les messages WebRTC ;
 - notifier les statuts d'upload.
-

10.3 Backend API

Responsabilités :

- authentication host ;
 - studios ;
 - sessions ;
 - participants ;
 - recordings ;
 - permissions ;
 - accès aux fichiers ;
 - dashboard ;
 - quotas plus tard ;
 - billing plus tard.
-

10.4 Upload Service

Responsabilités :

- recevoir les chunks ;
 - vérifier les chunks ;
 - reprendre un upload interrompu ;
 - calculer progression ;
 - signaler les chunks manquants ;
 - finaliser les pistes ;
 - déclencher les jobs de processing.
-

10.5 Object Storage

Responsabilités :

- stocker les chunks bruts ;
- stocker les pistes reconstruites ;
- stocker les exports finaux ;
- stocker les transcriptions ;
- stocker les sous-titres.

Les gros fichiers média ne doivent pas être stockés directement dans la base relationnelle.

10.6 Database

Responsabilités :

- utilisateurs ;
- studios ;
- sessions ;
- participants ;
- tracks ;
- chunks ;
- upload states ;
- jobs ;
- exports ;
- transcripts ;
- edit decisions.

La base de données stocke les métadonnées, les relations, les statuts et l'historique.

10.7 Workers

Responsabilités :

- assembler les chunks ;

- vérifier l'intégrité ;
 - synchroniser audio/vidéo ;
 - générer la vidéo finale ;
 - générer la transcription ;
 - générer les sous-titres ;
 - appliquer les cuts ;
 - produire les exports.
-

11. Contraintes de qualité

11.1 Qualité vidéo cible

La qualité cible du MVP est :

1080p stable

Le système doit viser :

- export final en 1080p ;
 - layout YouTube 16:9 ;
 - qualité locale supérieure à la qualité du live ;
 - adaptation du live selon réseau ;
 - priorité à la fiabilité du fichier final.
-

11.2 Résilience

Le système doit être testé contre les scénarios suivants :

- coupure réseau pendant recording ;
 - coupure réseau pendant upload ;
 - refresh de page ;
 - fermeture accidentelle ;
 - retour dans la session ;
 - upload incomplet guest ;
 - fichier partiellement uploadé ;
 - retry après erreur ;
 - serveur temporairement indisponible ;
 - worker processing en échec.
-

11.3 Observabilité

Le système doit permettre de savoir :

- combien de sessions sont en cours ;
- combien d'uploads sont incomplets ;
- combien de chunks échouent ;
- combien de jobs processing échouent ;

- temps moyen de génération export ;
 - erreurs WebRTC ;
 - erreurs upload ;
 - erreurs transcription ;
 - espace de stockage utilisé.
-

12. Ambition business

Djoko Studio peut commencer comme un outil personnel, mais doit être pensé dès le départ comme un SaaS individuel.

Modèle possible plus tard :

Free :

- nombre limité d'heures par mois
- watermark éventuel
- stockage limité
- exports limités

Starter – 5\$/mois :

- plus d'heures
- pas de watermark
- exports 1080p
- transcription limitée
- stockage plus long

Pro :

- plus de stockage
- plus d'heures
- transcription avancée
- édition texte
- screen share

Le pricing ne doit pas être basé uniquement sur l'accès au produit, mais sur les coûts réels :

- heures d'enregistrement ;
 - stockage média ;
 - transcription ;
 - processing vidéo ;
 - exports.
-

13. Pourquoi ce projet est pertinent pour apprendre l'architecture

Djoko Studio est un excellent projet d'apprentissage parce qu'il force à traiter de vrais problèmes de production.

Il permet d'apprendre :

- architecture frontend/backend ;
- WebRTC ;
- upload résilient ;
- stockage local navigateur ;
- object storage ;
- bases de données relationnelles ;
- modélisation de données ;
- workers asynchrones ;
- files d'attente ;
- traitement vidéo ;
- transcription IA ;
- édition vidéo ;
- déploiement multi-services ;
- observabilité ;
- sécurité ;
- migrations de données ;
- scalabilité.

Ce projet ne se limite pas à un CRUD.

Il oblige à penser comme un engineer production-ready.

14. Décisions d'architecture à documenter plus tard

Chaque décision importante devra être documentée dans un ADR.

ADR signifie **Architecture Decision Record**.

C'est un court document qui explique :

- le contexte ;
- les options envisagées ;
- la décision prise ;
- les conséquences positives ;
- les conséquences négatives.

Premiers ADR à prévoir :

ADR 0001 – Local-first recording
ADR 0002 – 1 host + 1 guest for MVP
ADR 0003 – Guest without account
ADR 0004 – Chunked upload with resume
ADR 0005 – Separate tracks for each participant
ADR 0006 – Object storage for media files
ADR 0007 – 1080p stable export target
ADR 0008 – Text-based editing architecture
ADR 0009 – Screen share integration strategy
ADR 0010 – Multi-tenant SaaS from the beginning

Ces ADR permettront de ne pas laisser l'IA prendre des décisions implicites sans justification.

15. Définition courte du projet

Djoko Studio est une plateforme web d'enregistrement d'interviews vidéo à distance, conçue pour produire des vidéos 1080p fiables même avec des connexions instables.

Le produit repose sur l'enregistrement local, l'upload résilient par chunks, les pistes audio/vidéo séparées, la reprise automatique après coupure, puis l'export final adapté à YouTube.

Le MVP est organisé en trois étapes :

- v0.1 : recording fiable ;
- v0.2 : transcription et édition par texte ;
- v0.3 : screen share intégré dans la vidéo finale.

L'objectif final est de créer un outil utilisable par des créateurs et entrepreneurs, tout en servant de projet d'apprentissage avancé en architecture, system design, bases de données, migrations, déploiement et scalabilité.